

Proposal to Apache Arrow to Extend PME

[Background and Motivation](#)

[Approach](#)

[End-to-end External Encryption and Decryption Flow](#)

[Configuration Management](#)

[Metadata Encryption](#)

[Changes to Codebase](#)

[Extension of Configuration and File Properties](#)

[Encryptor and Decryptor Class Abstraction](#)

[Creating Encryptors in InternalFileEncryptor](#)

[Creating Decryptors in InternalFileDecryptor](#)

[New External Encryptor and Decryptor classes](#)

[Minimizing Impact of Code Changes](#)

[Changes to Bindings for Supported Platforms](#)

[Proposed Changes to Parquet files, Metadata, and New Configuration](#)

[Enum Updates](#)

[Extending Modular Support for Encryption Algorithm Specification](#)

[Changes to Parquet files](#)

[Location of New Configurations](#)

[Integrating the New External DBPA Service in Arrow Applications](#)

[Behavior and Performance Impact on Arrow and Existing PME Functionality](#)

[Testing](#)

Background and Motivation

[Apache Arrow](#) is an open source platform for in-memory columnar data with support for multiple data formats, including [Apache Parquet files](#). Arrow is used in well-established industry applications and has a wide user base. Arrow supports [Parquet Modular Encryption \(PME\)](#), a feature to encrypt Parquet file data and metadata. PME allows for fine-grained access control by permitting different columns to be encrypted with distinct keys. While column data is selectively encrypted, it still allows efficient column filtering and predicate pushdown for efficient data selection during queries. PME ensures data remains protected while keeping access to data efficient.

This proposal details a project where PME's capabilities are extended by using an external third-party encryption mechanism for column chunk encryption operations instead of the internally-defined encryptors. Arrow applications can specify an additional configuration to indicate the use of an external encryption service for a specific column, along with attributes specific to the external encryption

functionality. We refer to this external application as the Data Batch Protection Agent (DBPA or External DBPA).

An external encryptor can expand the capabilities of the encryption functions beyond processing a column chunk as one simple unit. A context aware encryption function can apply more robust access control and have finer-grained checks based on the user of an Arrow application. Also, datatype- and/or value-specific encryptions can be applied to the items of the data chunks individually. This allows more flexible and robust encryption operations to be utilized and still be fully integrated on the Arrow's Parquet file handling.

Furthermore, this extension also proposes a further step in module encryption: the ability to specify different encryption algorithms to be used for each encrypted column. Currently, Arrow only supports using different keys on columns or encrypting a subset of columns. This project aims to implement fully modular encryption on a per-column basis.

This enhancement is vital for addressing varying security needs within data files. Since data sensitivity differs by column (e.g., personally identifiable information vs. non-sensitive data), the ability to assign distinct encryption algorithms on a per-column basis allows for specialized, stronger protection for the most sensitive fields. This granular control ensures optimal security configuration and better resource alignment across the entire dataset.

Benefits to the Arrow infrastructure:

- More sophisticated and resource intensive encryption algorithms can be used by offloading the task to an external service optimized for this purpose. This reduces the existing resource footprint of Arrow in production setups.
- Modularity of the encryption operations. New encryption algorithms can be made available seamlessly without the need of Arrow production releases. The community is able to implement data encryption services of their own and integrate with Arrow by implementing the specified interface.

Benefits to the Arrow applications:

- Applications can specify additional parameters for external encryption functions to customize the operation. These parameters can be used to implement additional access control checks or customize the encryption behavior based on user, geography, or other application context values.
- Makes encryption operations more secure by potentially delegating the encryption key lookup to the external service preventing the keys from traveling through the wire.

This document presents the design and rationale for the changes [implemented in a fork of the Apache Arrow codebase](#), how the new configuration for an external encryption service at Arrow application's level works, and the expected impact of these changes.

Approach

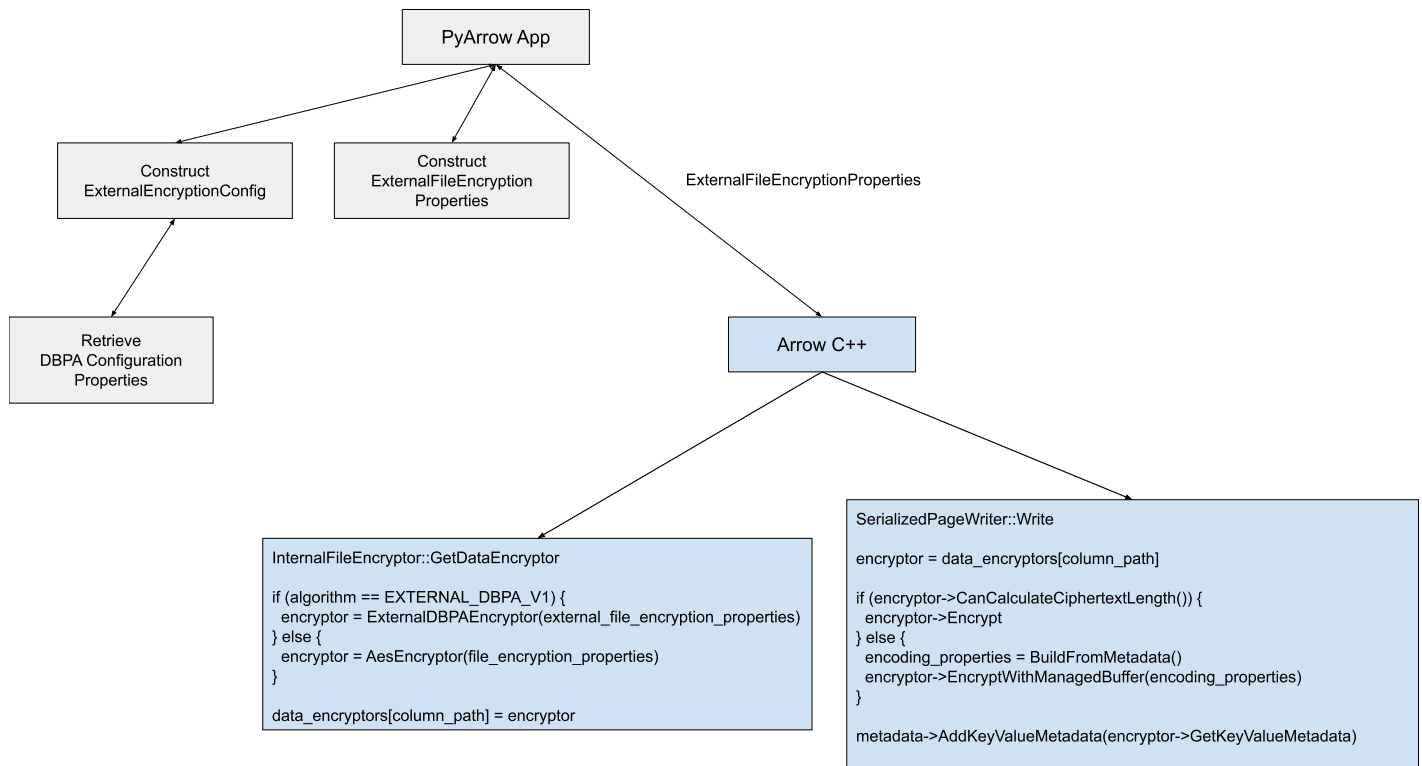
The changes described in this document were designed to (1) follow the class design and configuration patterns in the Apache Arrow codebase to ease integration and follow the “Arrow style” whenever possible, (2) maintain the existing encryption flows unchanged to minimize the impact of introducing the service-based encryption on current functionality, and (3) reduce changes to the Parquet file format and do it as extensions of dictionaries on the file metadata if needed.

To start, we created [subclasses for both the encryptor and decryptor configuration and file properties](#). This approach allows us to reuse Arrow's existing infrastructure for handling encryption and decryption via its [CryptoFactory](#) class. These external subclasses inherit properties from the existing classes but add fields to specify per-column encryption properties and include the necessary information for Arrow to interface with the External DBPA encryptor and decryptor (via a shared library, i.e. *.dll or *.so files).

The implementation of the new encryptor subclass that supports the external service abstracts all details related to the service connection, network sessions, and authentication from Arrow. This ensures that the Arrow codebase remains agnostic of any external encryptor implementation details. This was achieved by developing separate artifacts that are dynamically linked when external encryption is configured in the application. This separation also decouples the release cycles of the Arrow and SDK implementations, making it easier to switch between different SDK implementations if needed.

End-to-end External Encryption and Decryption Flow

A high level overview of the proposed solution is shown in the following diagrams. For the rest of this document, we will use PyArrow at the application level as an example.



The application must be responsible for knowing how to access the DBPA configuration properties, which will allow Arrow to work with the external code. This is used to create an instance of the `ExternalEncryptionConfiguration`, where the information on which columns to encrypt is stored. As is the expected behavior, the `CryptoFactory` will then use this external configuration to create the `ExternalFileEncryptionProperties`.

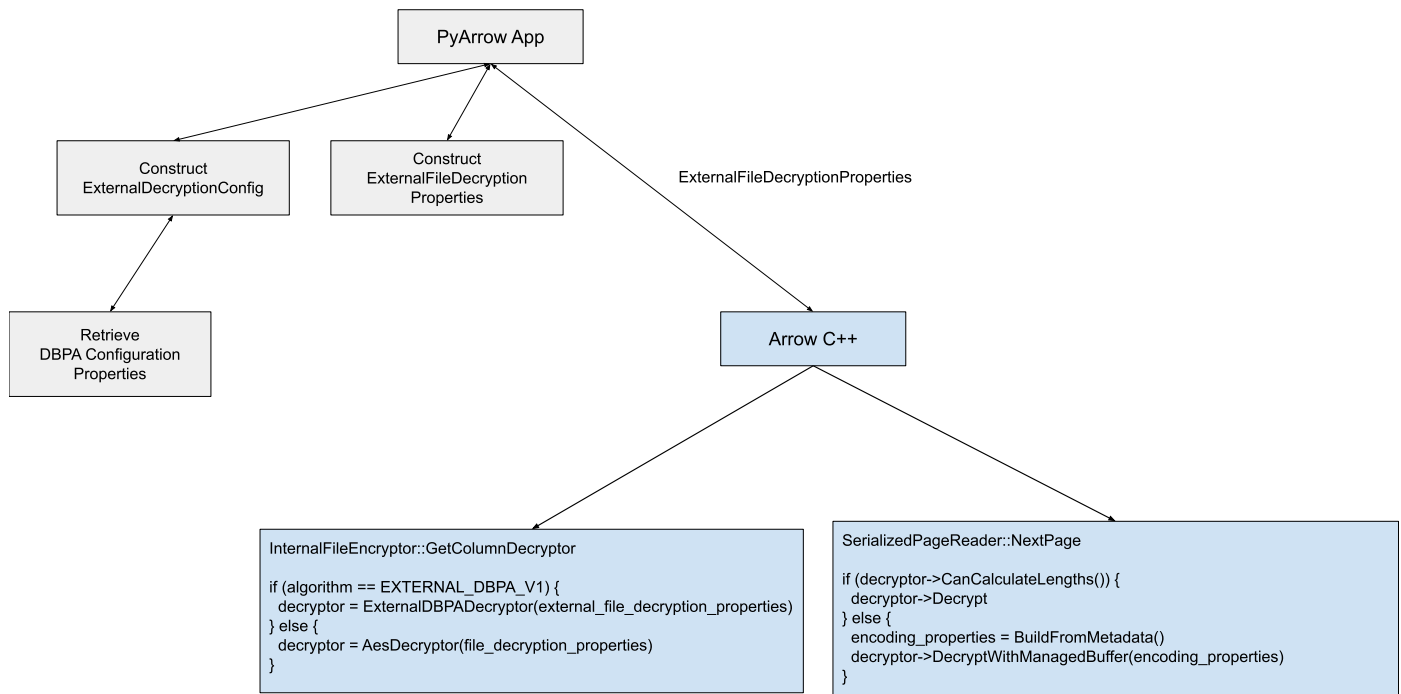
Once Arrow C++ is called, it receives the `ExternalFileEncryptionProperties`. This includes both the identifier of the external encryption service to use and any additional encryption parameters required.

Before writing to a Parquet file, Arrow initializes the necessary writers for each column. These writers are configured based on the column schema and any specified encryption properties. If encryption is required, Arrow creates a corresponding encryptor for each column and stores it in a map for later use. We modified this logic such that the type of encryptor stored in the map depends on the specified encryption algorithm *for a specific column*. When the algorithm is `EXTERNAL_DBPA_V1`, an instance of `ExternalEncryptor` is stored. In all other cases, the existing `AesEncryptor` is used.

The code changes during the actual encryption process are minimal. We first retrieve the column's encryptor from the map. Then, we added a new entry point [for encryptors who cannot calculate the ciphertext length before the actual encryption takes place](#), so we check which version of the encrypt call to make.

We also update the page's metadata to include any information that the encryptor sends back as its own metadata that it might need in order to decrypt.

The decryption flow closely mirrors the encryption flow. The application must again know how to obtain the DBPA configuration properties, build the ExternalEncryptionConfiguration and ask the CryptoFactory for the ExternalFileDecryptionProperties.



The key difference is that Arrow reads the encryption algorithm directly from the Parquet file metadata. Based on this information, Arrow can proceed to create the appropriate decryptor for each column, using the same logic as in encryption but with a new check to select the correct implementation.

During reading, the decryptor that was created for each column must again go through a check of whether it is able to calculate ciphertext and plaintext lengths before the actual decryption happens.

Configuration Management

In order to support external encryptors, we need to send additional parameters from the application layers into Arrow. We [have extended file encryption and decryption properties](#) to include:

- A configuration properties map that allows the new classes to obtain information on how to instantiate the external encryptor, and additional parameters which may be needed by the encryptor itself.
- An application context dictionary (user provided metadata relevant to the encryptor).

These extensions have been implemented in Arrow C++ and also in the PyArrow (Cython) bindings so they can be used from the top-level applications. Adding these extensions did not modify the behavior of the existing encryption APIs.

Metadata Encryption

Our current proposal focuses exclusively on data encryption. We do not implement metadata encryption at this stage.

Changes to Codebase

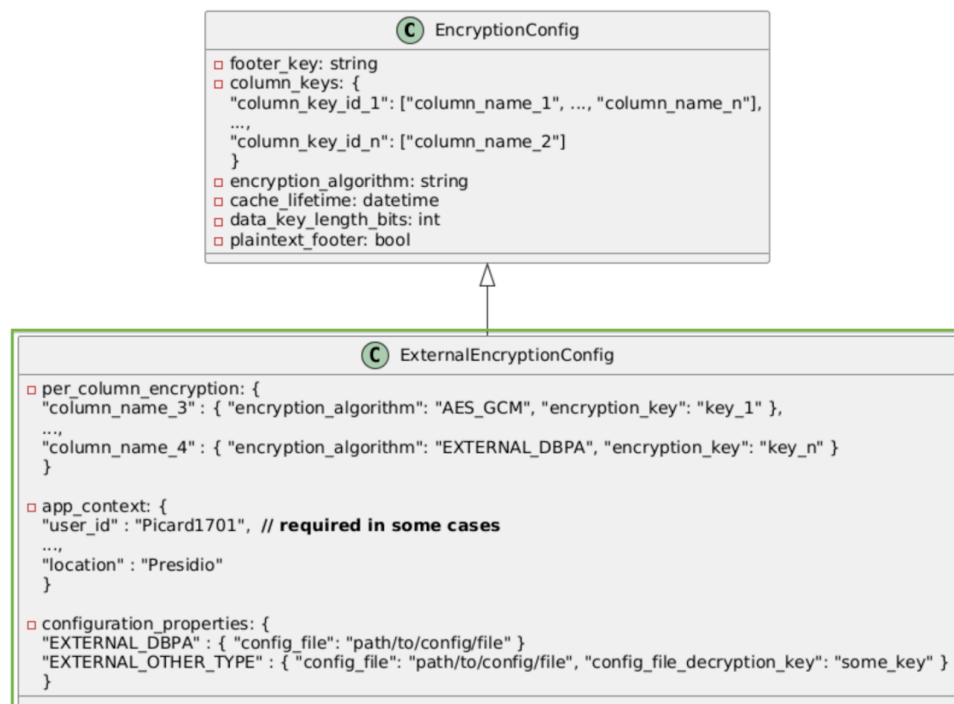
A list of the files touched during this project can be found in the [Protegrity Arrow Code Map document](#). It provides a high level overview of the changes made to all files involved in this project.

Extension of Configuration and File Properties

To support external encryptors, we provide a new `ExternalEncryptionConfig` class (which extends the current `EncryptionConfig`) and its corresponding `ExternalFileEncryptionProperties`. These classes allow developers to specify:

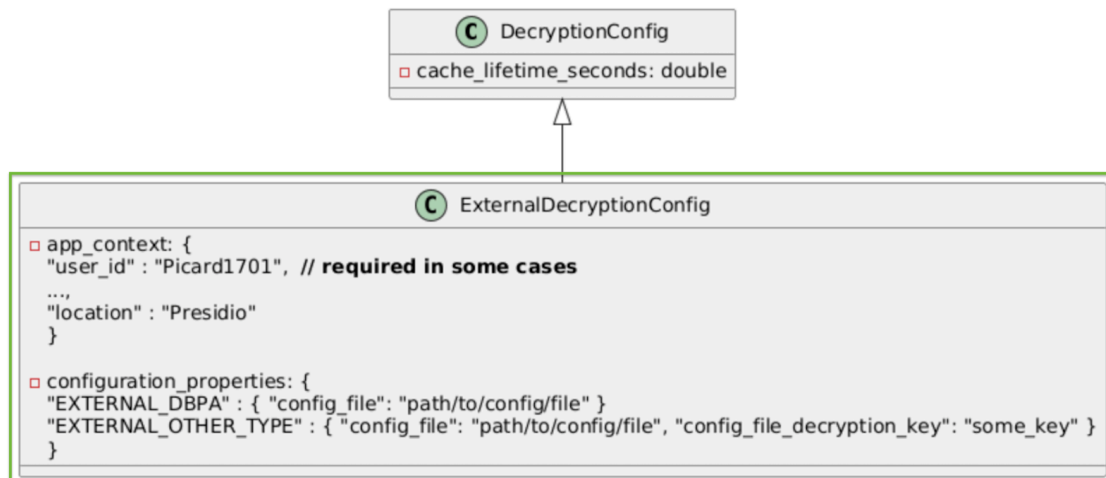
- How to connect with the external service
- External service specific parameters, in the form of an application context
- Most importantly, allow developers to specify *per-column* encryption configuration

Please see the [Encryption section of the External Config and Properties Discussion document](#) for a detailed description of the new fields added. The new classes are:





Similarly, we extend the same information available in the [Decryption configuration and file properties](#). The new classes are:

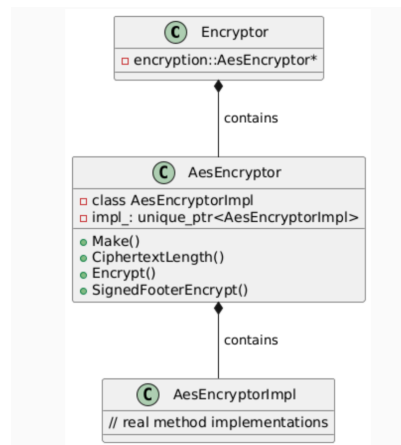




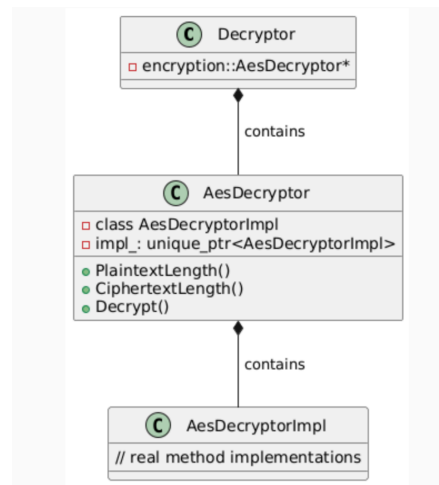
These new classes are defined in both Arrow C++ and in the PyArrow bindings (Cython) code.

Encryptor and Decryptor Class Abstraction

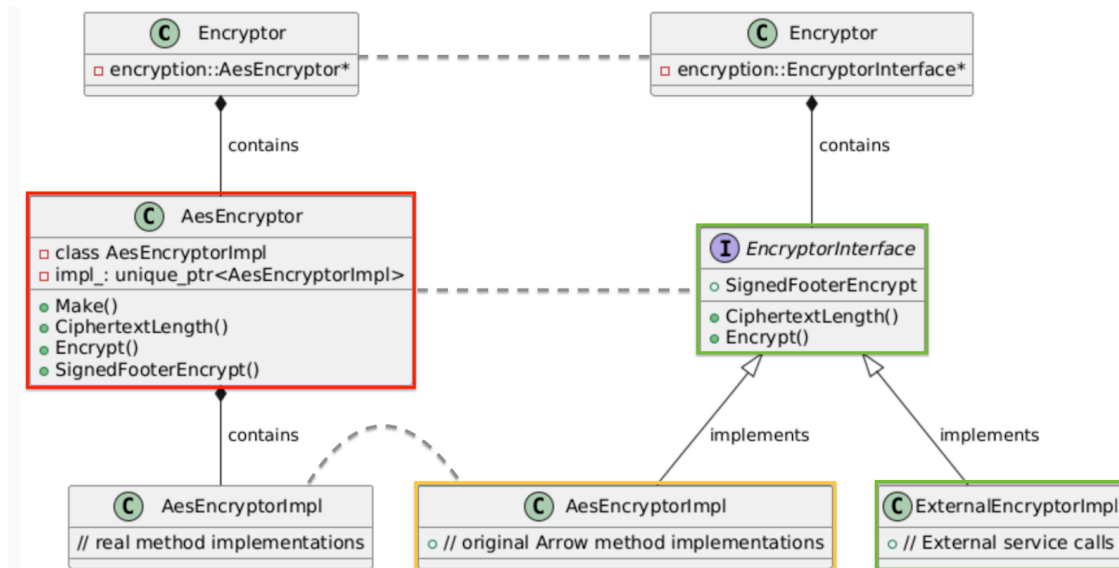
Originally, Arrow only supported AES encryption and decryption algorithms. As a result, the codebase was tightly coupled to that implementation, with AES-specific naming and types propagated throughout. The [Encryptor](#) wrapper class contained a reference to an [AesEncryptor](#), which in turn had a pointer to its implementation ([AesEncryptorImpl](#)).



Likewise, the [Decryptor](#) wrapper class contains a reference to an [AesDecryptor](#), with a pointer to its implementation ([AesDecryptorImpl](#)).

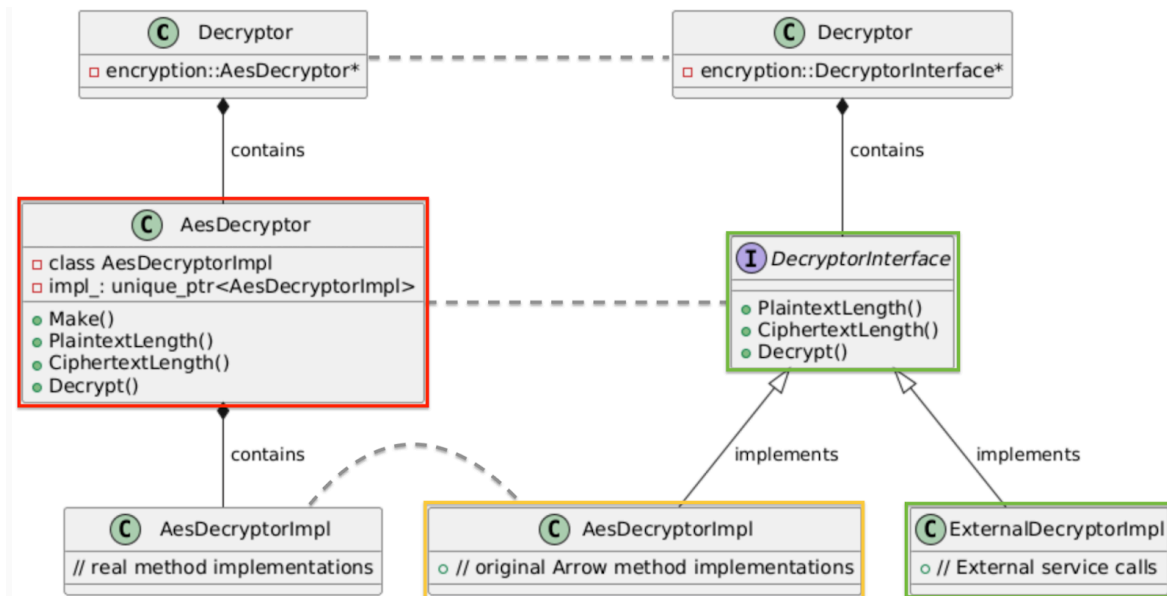


One important contribution made by this project is the decoupling of the AES algorithm from default encryption and decryption within Arrow. This refactor introduced an [EncryptorInterface](#), which lets us keep the separation between the Encryption wrapper and the specific implementation, while allowing future algorithms to be integrated without modifying the wrapper or introducing further coupling. We refactored the [AesEncryptor](#) to inherit from this new interface instead.



The original Encryptor classes are shown on the left. On the right, we see the Encryptor wrapper now contains a pointer to the new **EncryptorInterface** class. This interface defines the methods that all encryptors must implement. The original **AesEncryptor** class is removed, and now the **AesEncryptorImpl** class implements the **EncryptorInterface**, as does the new **ExternalEncryptorImpl**.

The class structure for the Decryptor is identical, hence so are the changes for the new [DecryptorInterface](#). Note that in both cases, the separation between the wrapper and the corresponding implementations is preserved.

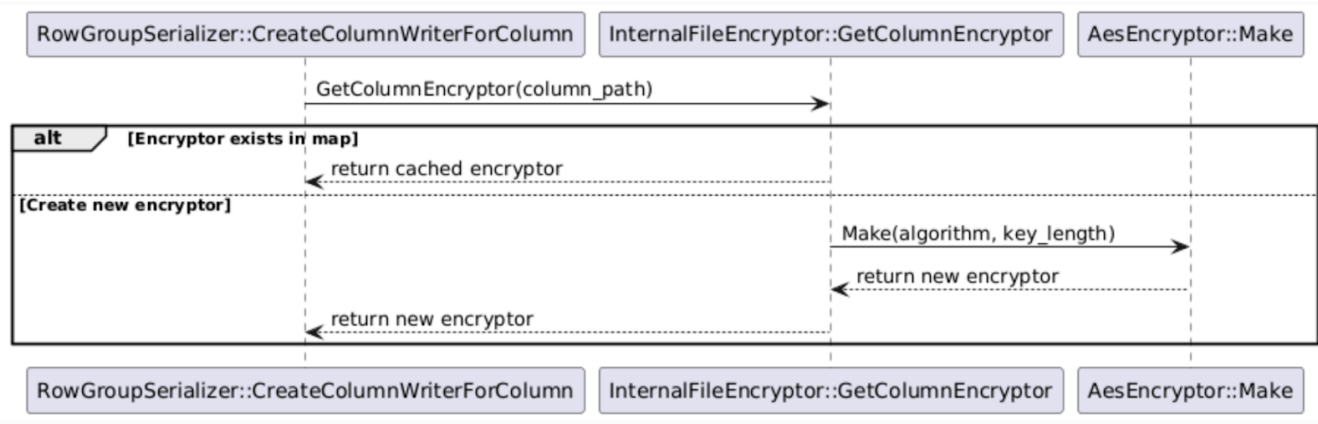


This refactor needed further decoupling from the AES encryption and decryption models, as not every algorithm is capable of knowing the ciphertext and plaintext length in advance of an encryption or decryption operation. We discuss this limitation at length in the [Ciphertext Length document](#), and propose a solution where each encryptor and decryptor must be able to signal whether it is capable of making these calculations.

Depending on this, the column writer and reader will call the correct encrypt and decrypt method (the traditional version, or a new version which manages and resizes its buffers as needed). We discuss later in this document the performance implications of this change, which are minimal.

Creating Encryptors in InternalFileEncryptor

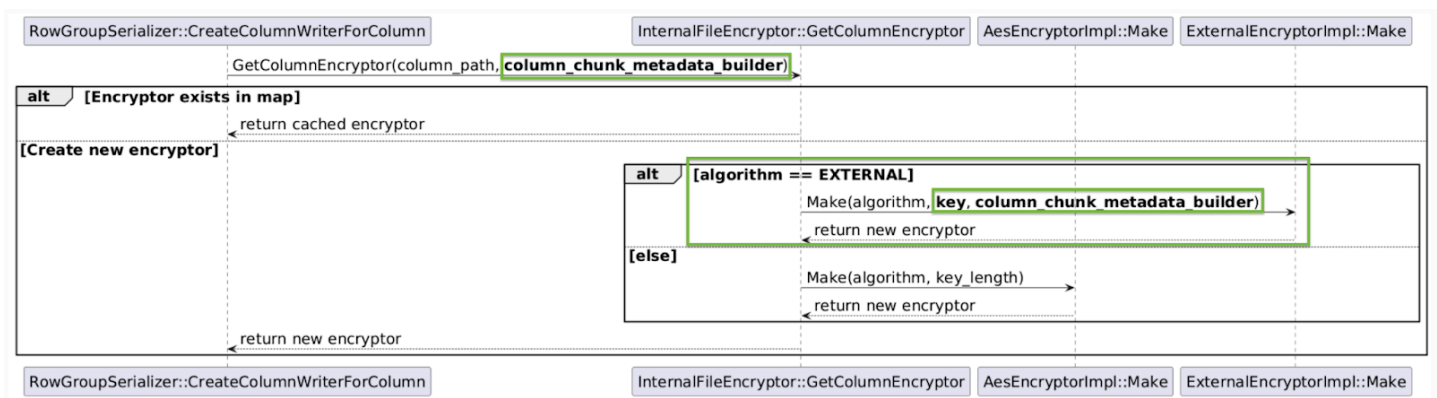
Through our investigation of Arrow's encryption flow, we found that each column chunk is handled by the [RowGroupSerializer](#) class, which is responsible for creating a column writer per column. If encryption is enabled, this process also instantiates an encryptor for the column and stores it in a map for later use during data encryption. This flow is represented in the following diagram:



[This stage turned out to be the ideal insertion point for our extension.](#) Here, we can inspect the encryption algorithm specified and, when appropriate, instantiate our custom `ExternalEncryptor`.

The only additional requirement is access to the `ExtendedFileEncryptionProperties`, so we can supply the external encryptor with the configuration it needs.

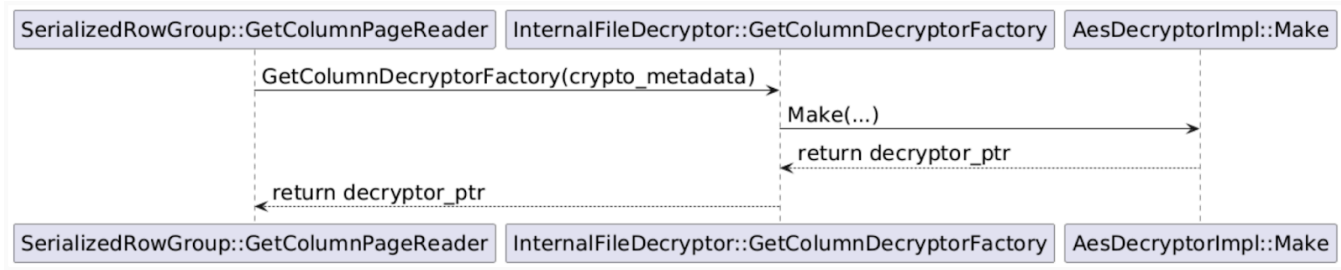
To support this, we updated some method signatures in both `RowGroupSerializer` and [InternalFileEncryptor](#) to include a reference to the [ColumnChunkMetadataBuilder](#). This object encapsulates relevant column metadata (such as schema details, compression settings, and encryption configuration) and provides the necessary context for constructing the appropriate encryptor instance.



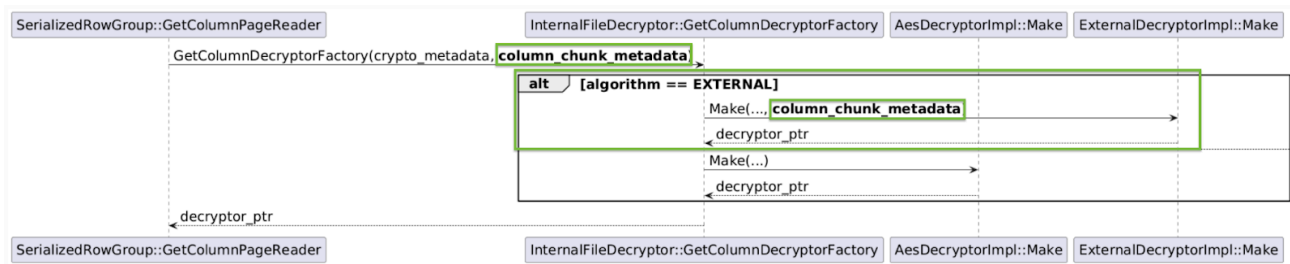
The modified sequence diagram shows the new parameters sent to the internal methods, as well as the check for the new type of algorithm.

Creating Decryptors in InternalFileDecryptor

When reading an encrypted Parquet file, the flow is similar in that each column chunk is read by a [SerializedRowGroup](#), which is responsible for creating a column page reader. In this code a decryptor is created and its pointer is returned for later use. The original flow is:



We [updated the decryption logic to mirror the encryptor changes](#). Specifically, we introduce a check on the encryption algorithm used to determine which type of decryptor factory to create.



As with encryption, the external decryptor requires additional configuration, which is expected to reside in the column chunk metadata. To support this, we extended the relevant method signatures in `InternalFileDecryptor` to propagate access to the [ColumnChunkMetaData](#), enabling the decryptor to extract the information it needs.

New External Encryptor and Decryptor classes

We [added new classes for the external encryptor and decryptor](#). The Data Batch Protection Agent (DBPA) is fully independent from Arrow in its encrypt and decrypt operations, but these classes do implement the new Encryptor and Decryptor Interfaces that were described above. This [document contains a full description of DBPA implementation, contract, and inner workings](#).

In relation to DBPA, it is important to call out the following in:

- The new encryptor and decryptor are designed to perform *value-based* encryption, as opposed to block-based (which is AES' behavior). As such, when the DBPA code receives a data page that it must encrypt, it will decompress it (if necessary), decode it (if possible), and then parse the

values one by one in the column block so that it may perform individual encrypt (or decrypt) operations over those values.

- This sequence of events is inherently more expensive than a regular block based operation over a compressed block of the data. The code that performs this operation is highly optimized such that performance can be as quick as possible.
- Because the column chunk on the DBPA side needs to be uncompressed and parsed, DBPA must receive additional metadata information from Arrow. This includes the data type, compression type, and datatype length.
- For this purpose, we created an additional class, the [EncodingProperties](#), that encapsulates a column's metadata information such as data type, page encoding, page version, repetition level encoding, and level byte length that help the DBPA code to perform highly efficient byte level operations in order to uncompress and parse every value present in the column chunk.
- Before every call to [encrypt](#) or [decrypt](#), we construct the EncodingProperties for the column chunk and send it over to the DBPA agent.

Minimizing Impact of Code Changes

By identifying the exact locations where encryptors and decryptors are created, we isolate the logic that needs to handle the new encryption algorithm.

This targeted modification ensures that the broader encryption and decryption flows used during Parquet file read and write remain largely unchanged. The introduction of clean interface abstractions further reinforces this boundary, allowing new implementations to integrate seamlessly without impacting existing behavior.

Changes to Bindings for Supported Platforms

In order to use the PME Extension code, Arrow bindings must implement the new external versions of the encryption and decryption configs and external file encryption and decryption properties. These bindings convert to the proper C++ classes that are sent to Arrow.

We have implemented these changes already in PyArrow (Cython) bindings. No functional logic was added to this code other than the creation of the corresponding C++ classes needed:

- [ExternalEncryptionConfiguration](#)
- [ExternalDecryptionConfiguration](#)
- [ExternalFileEncryptionProperties](#)
- [ExternalFileDecryptionProperties](#)

Currently Python is the only language with supported bindings. Any other language that wants to be able to use the PME extension needs to make changes in their own binding code. If they

work with the C++ version of Apache Arrow, this should be enough. However, other languages (such as Java) which have their independent Arrow implementation, will be required to fully implement the PME extension.

Proposed Changes to Parquet files, Metadata, and New Configuration

Enum Updates

We extended the existing [ParquetCipher::type](#) enum to include a new value representing the external encryption service.

```
struct ParquetCipher {  
    enum type { AES_GCM_V1 = 0, AES_GCM_CTR_V1 = 1, EXTERNAL_DBPA_V1 = 2 };  
};
```

This new enum value (`EXTERNAL_DBPA_V1`) is now used throughout the codebase to determine whether to instantiate the standard or external encryptor/decryptor implementations. Naturally, updating the enum required corresponding updates across all Arrow language bindings to ensure consistency.

Additionally, the [C++ Thrift](#) definitions were updated to ensure that the new enum value can be properly serialized and deserialized as part of the Parquet file metadata.

Extending Modular Support for Encryption Algorithm Specification

A key enhancement of this proposed project is the ability to specify different encryption algorithms for different columns. As we propose an extension wherein an external encryptor can be used to encrypt values, it is a natural use case for developers to decide which critical columns must use an external encryptor, and whether other columns can remain encrypted using more traditional implementations.

We [have already discussed](#) how the `ExternalEncryptionConfig` and the `ExternalFileEncryptionProperties` handled this new per-column requirement. A new *optional* `ParquetCipher` field was added to the [ColumnEncryptionProperties](#) class that can let us handle this extension. If present, this field is populated in the [CryptoFactory method that constructs the ExternalFileEncryptionProperties](#). From this point on, the information is propagated to the rest of the code, where it can be used (when present) to create the specific type of encryptor needed by that column.

In the next section, we discuss where this field is persisted in the Parquet file metadata, allowing us to also create the correct type of decryptor for the column as needed.

Changes to Parquet files

Parquet files store metadata in their footers that is critical for interpreting and decrypting the data during reads. A key design question is whether this metadata needed to be extended to support external encryption.

At this stage, no structural changes to the Parquet metadata were necessary. The file format already stores the file-level encryption algorithm identifier in the footer, and we introduced an additional field in the [ColumnCryptoMetaData](#) that allows us to specify an encryption algorithm *per-column*.

As previously mentioned, we also allow encryptors and decryptors to propagate additional metadata to save after their operations conclude. As these values are added to the existing [key_value_metadata within ColumnMetadata](#), the structure of the Parquet files remains unchanged.

Location of New Configurations

We introduce a new type of configuration: the [configuration_properties](#). This information is needed to allow users to specify how to instantiate and configure any type of external encryptor they wish to use. This data is not intended to be stored in the Parquet file, or used by Arrow in any way.

Instead, the information is given to the ExternalDBPA adapters, who will know how to process and use this information. These properties are a map, containing an entry for each type of encryptor. The only processing done by Arrow is selecting the configuration that corresponds to the service key (right now, only EXTERNAL_DBPA_V1).

For security purposes, it is important that no explicit information (such as credentials) are stored directly in the map. Rather, a path to a configuration file should be provided and the ExternalDBPA adapters should know how to access and process the file information.

The other new configuration present is the [app_context](#). This dictionary will be forwarded to the external service as is, with no Arrow processing whatsoever.

Integrating the New External DBPA Service in Arrow Applications

For a top-level Arrow application, the existing flows for writing and reading encrypted Parquet files remain largely unchanged. The only difference comes in the fact that:

- An [ExternalEncryptionConfig](#) object must be created, and it must contain per-column encryption specifications. The same applies to the [ExternalDecryptionConfig](#).

- The new `CryptoFactory` class method that creates an [ExternalFileEncryptionProperties](#) object must be called. For decryption, the new method that creates the [ExternalFileDecryptionProperties](#) is called.
- The call to the `WriteTable` or `ReadTable` methods now receive the external objects created above. No changes to the method signatures of `WriteTable` or `ReadTable` are required.

Please refer to the [use_external_dbpa_encryption.py](#) script for a complete, working example of what PyArrow code would look like when using an external service for file encryption and decryption.

Behavior and Performance Impact on Arrow and Existing PME Functionality

We have minimized the impact of our solution on existing Arrow PME functionality. By intercepting the code where encryptors are constructed, we avoid major alterations to the core write/encrypt and read/decrypt flows. The new encryptor classes implement an interface compatible with the current `AesEncryptor`, ensuring that encryption logic remains consistent.

In addition, our approach does not require changes to the structure of Parquet files. We have added new metadata that is serialized in the Parquet file, including the new encryption algorithm enum value in the [ColumnCryptoMetaData](#) object. However, the structure of the Parquet file remains unchanged, and any additional metadata has been placed in already existing structures.

We have also followed established Arrow patterns: any failure during encryption or decryption will cause the process to halt immediately. To avoid any potential data corruption, the external service never returns partial results—any error condition will result in a surfaced failure, preserving the integrity of the data and the operation.

The primary risk introduced by this solution is the dependency on an external encryptor, which may be powered by an external service. This inherently adds potential for latency, bottlenecks, and operational failures when the service is unavailable or unresponsive. We have performed basic benchmark testing to ensure performance is optimal. Even when the encryptor is fully local (i.e. no over-the-network requests), the switch to using per-value encryption (instead of AES' per-block) represents an increase in encryption and decryption latency times. The trade-off is the availability of a more robust, feature rich, and secure encryption mechanism for better data security.

[This document summarizes the findings of our benchmark tests.](#) We are happy to report that the design of our project resulted in minimal latency overhead on Arrow when interfacing with an external encryptor. We specifically measured the impact *within Arrow* of calling an external encryptor or decryptor, and of using the managed buffer solution. The cost of these operations is only ~0.2% of the overall time involved in the encryption/decryption operations.

The implemented external encryption and decryption service operates on a per-value basis. As expected, the majority of the cost within the external libraries is spent in the logic that decompresses the data and operates on a single value at a time. When compared with AES encryption (which can handle compressed blocks of data at a time), we were still able to maintain performance at an acceptable overhead:

String Data (i.e. variable-sized data) (First and last names)

Encryption	Decryption	# of Data Pages
DBPA Δ: +1,565 ms (+76.4%) (AES: 2,049 ms) (DBPA: 3,614 ms)	DBPA Δ: +1,834 ms (+182.7%) (AES: 1,004 ms) (DBPA: 2,838 ms)	173 pages (~57.8k values/page)

INT32 Data (i.e. fixed-sized data)

Encryption	Decryption	# of Data Pages
DBPA Δ: +161 ms (+45%) (AES: 357.8 ms) (DBPA: 518.8 ms)	DBPA Δ: +222 ms (+168.2%) (AES: 131.7 ms) (DBPA: 353.7 ms)	20 pages (500k values/page)

It is important however, that any new external encryptor and decryptor be mindful of its own implementation and optimizes code on its end to minimize impact. Please see the [cost analysis document](#) for details on performance times and code optimizations done for our external implementation.

Testing

Please find details for the implemented test plan in [this document](#).

<>